Sous-partie A — Analyse fonctionnelle

1. Présentation générale du système

La plateforme développée a pour objectif de gérer et superviser les livraisons de colis d'une startup logistique marocaine opérant dans plusieurs villes telles que Fès, Casablanca, Rabat, Marrakech et Tanger. Le système doit permettre le traitement en temps réel des commandes clients, le suivi des différentes étapes de livraison ainsi que la production de statistiques de performance pour les responsables logistiques.

L'architecture choisie repose sur un modèle orienté événements (*Event-Driven Architecture*). Chaque changement d'état d'une commande génère un événement métier transmis dans un système distribué simulé en Python. Cette approche permet de découpler les différents composants, d'améliorer la scalabilité du système et de garantir une meilleure tolérance aux pannes.

Le système est conçu pour supporter plusieurs centaines d'événements par minute tout en maintenant un suivi précis des commandes. Les événements sont partitionnés selon la ville afin de répartir efficacement la charge de traitement et d'optimiser les accès aux données.

Le cycle de vie d'une commande passe par plusieurs états métier : création, confirmation, préparation, expédition, livraison, annulation ou échec de livraison. Chaque transition produit un événement transmis aux différents composants consommateurs du système.

---

2. Acteurs du système

Le premier acteur du système est le service synchrone de réception des commandes. Ce composant reçoit les requêtes des clients et vérifie immédiatement la validité des données reçues. Lorsqu'une commande est acceptée, un événement `order_created` est généré. En cas de données invalides, le système retourne directement une erreur au client.

Le second acteur est le producteur d'événements métier. Ce composant transforme les actions métier en événements distribués vers le broker partitionné. Il génère notamment les événements :

- `order_confirmed`
- `order_prepared`
- `order_dispatched`
- `order_delivered`
- `order_cancelled`
- `delivery_failed`

Le broker partitionné constitue le cœur de l'architecture distribuée. Il reçoit les événements publiés par les producteurs et les répartit selon une clé de partitionnement basée sur la ville (`city`). Cette organisation permet un traitement parallèle des données.

Deux consumers spécialisés consomment ensuite les événements publiés. Le Consumer A est responsable du stockage persistant des événements ainsi que de certaines mises à jour métier. Le Consumer B est chargé de l'analyse temps réel, du calcul des indicateurs de performance et de la détection des anomalies comme les retards importants ou les livraisons échouées.

Enfin, les responsables logistiques consultent les informations via un tableau de bord texte interactif permettant de visualiser les statistiques globales, les anomalies et les performances par ville.

---

3. Flux de données du système

Le premier flux concerne la communication entre le client et le service de réception des commandes. Ce flux est synchrone car le client doit recevoir immédiatement une confirmation d'acceptation ou un message d'erreur concernant sa commande. Une validation immédiate est nécessaire afin de garantir l'intégrité des données saisies.

Une fois la commande validée, les événements métier sont transmis de manière asynchrone entre le service métier, le producteur d'événements et le broker partitionné. Ce choix permet de découpler les composants et d'éviter qu'un ralentissement dans un composant bloque l'ensemble du système.

Le broker transmet ensuite les événements vers plusieurs consumers spécialisés. Ce flux est également asynchrone car plusieurs composants doivent pouvoir traiter simultanément les mêmes événements sans dépendance directe entre eux. Le Consumer A traite principalement le stockage persistant tandis que le Consumer B réalise les calculs analytiques et la détection d'anomalies.

Les flux liés aux erreurs métier sont également pris en compte. Les erreurs de validation de commande sont traitées de manière synchrone car elles nécessitent une réponse immédiate au client. En revanche, les événements `delivery_failed` sont propagés de manière asynchrone afin de permettre leur traitement par les modules d'analyse et de supervision sans interrompre le reste du système.

Les données persistantes sont ensuite sauvegardées dans un stockage partitionné sur disque organisé par ville. Cette structure permet d'optimiser les recherches et les traitements analytiques futurs.

---

4. Justification des choix synchrones et asynchrones

Le choix entre communication synchrone et asynchrone dépend principalement de trois critères étudiés en cours :

- le besoin de confirmation immédiate,
- la nature du traitement,
- le nombre de récepteurs concernés.

Les flux synchrones sont utilisés lorsqu'une réponse immédiate est indispensable. C'est notamment le cas lors de la création d'une commande où le client doit savoir instantanément si la requête a été acceptée ou rejetée.

Les flux asynchrones sont privilégiés pour les traitements distribués et les événements métier. Les événements peuvent être consommés indépendamment par plusieurs modules sans bloquer le système principal. Cette approche améliore les performances globales, facilite la montée en charge et augmente la résilience face aux pannes.

L'utilisation d'un broker partitionné permet également d'assurer une meilleure répartition de charge et de traiter plusieurs flux simultanément selon les villes concernées.